

dgiot_lite

部署手册

光储充微电网物联网平台

版本号: V1.0

日 期: 2026 年 6 月

适用系统: Ubuntu 20.04+ / CentOS 7.9 / Windows 10+

迪格 (杭州) 物联科技有限公司

目录

1	环境要求	2
1.1	硬件配置	2
1.2	软件依赖	2
2	快速部署（Docker，推荐）	2
2.1	一键启动	2
2.2	验证部署	3
2.3	停止服务	3
3	裸机部署（Linux/Windows）	3
3.1	安装 Python 依赖	3
3.2	配置	4
3.3	初始化数据库（可选）	4
3.4	启动平台	5
3.5	设置开机自启（Linux）	5
4	数据库部署	6
4.1	PostgreSQL	6
4.2	TDengine	6
4.3	不装数据库？	6
5	协议模拟器	7
5.1	启动模拟器	7
5.2	模拟设备清单	7
6	DG-IoT 联动配置（可选）	7
6.1	方法一：管理后台配置	8
6.2	方法二：API 配置	8
6.3	验证联动	8
7	部署验证清单	9
8	常见问题	9
8.1	端口被占用	9
8.2	数据库连接失败	9
8.3	模拟器启动报错	10
8.4	如何更新平台	10

1 环境要求

1.1 硬件配置

表 1: 服务器配置建议

资源	最小	推荐	生产
CPU	2 核	4 核	8 核
内存	4 GB	8 GB	16 GB
磁盘	20 GB	100 GB SSD	500 GB SSD
网络	百兆	千兆	千兆双网卡

1.2 软件依赖

软件	版本要求	用途
Python	≥ 3.10	平台运行环境
PostgreSQL	≥ 15（可选）	关系数据库
TDengine	≥ 3.0（可选）	时序数据库
EMQ X	≥ 5.0（可选）	MQTT Broker
Docker	≥ 24.0（可选）	容器化部署

说明：PostgreSQL、TDengine、EMQ X 均为可选依赖。未安装时，平台自动降级为 SQLite 单机模式，零外部依赖即可运行。

2 快速部署（Docker，推荐）

2.1 一键启动

```
# 进入部署目录
cd dgiot_lite/scripts

# 启动全部服务
docker-compose up -d
```

启动后自动创建 4 个容器：

容器	端口	说明
iot-platform	8000	dgiot_lite 核心服务
postgres	5432	PostgreSQL 15
tdengine	6030/6041	TDengine 时序库
emqx	1883/18083	MQTT Broker

2.2 验证部署

```
# 检查容器运行状态
docker-compose ps

# 检查 API 健康状态
curl http://localhost:8000/api/health
# 返回: {"status":"ok","version":"1.0.0",...}

# 打开管理后台
# 浏览器访问: http://localhost:8000
```

2.3 停止服务

```
docker-compose down          # 停止
docker-compose down -v       # 停止并删除数据卷
```

3 裸机部署 (Linux/Windows)

3.1 安装 Python 依赖

```
# 克隆仓库
git clone git@git.iotn2n.com:dgiot/dgiot_lite.git
cd dgiot_lite

# 创建虚拟环境 (推荐)
python -m venv venv
source venv/bin/activate      # Linux
venv\Scripts\activate        # Windows
```

```
# 安装依赖
pip install -r requirements.txt
```

3.2 配置

编辑项目根目录下的 config.yaml:

```
title: "光储充微电网物联网平台"
version: "1.0.0"
host: "0.0.0.0"
port: 8000

db:                                     # PostgreSQL (无则自动 SQLite)
  host: "127.0.0.1"
  port: 5432
  user: "postgres"
  password: "postgres"
  database: "iot_platform"

tdengine:                             # TDengine (无则自动 SQLite)
  host: "127.0.0.1"
  port: 6030
  user: "root"
  password: "taosdata"
  database: "iot_telemetry"

mqtt:                                  # MQTT Broker (推送用)
  host: "127.0.0.1"
  port: 1883
  username: ""
  password: ""
```

最简配置: 无需修改任何参数。平台启动后自动使用 SQLite 降级模式。

3.3 初始化数据库 (可选)

```
python scripts/init_db.py
```

此脚本自动创建 PostgreSQL 表结构 + TDengine 超级表。如果数据库不可用, 将自动降级 SQLite, 此步骤可跳过。

3.4 启动平台

```
python run.py
```

输出示例:

```
INFO:      Uvicorn running on http://0.0.0.0:8000
INFO:      [sqlite] 降级模式已启用
INFO:      [collector] 启动完成, 0 个设备
INFO:      [main] dgiot_lite V1.0.0 启动完成
```

Windows 用户: 也可双击 start_platform.bat。

3.5 设置开机自启 (Linux)

```
# 创建 systemd 服务
sudo tee /etc/systemd/system/dgiot_lite.service << 'EOF'
[Unit]
Description=dgiot_lite IoT Platform
After=network.target

[Service]
Type=simple
User=dgiot
WorkingDirectory=/opt/dgiot_lite
ExecStart=/opt/dgiot_lite/venv/bin/python run.py
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

# 启用
sudo systemctl daemon-reload
sudo systemctl enable dgiot_lite
sudo systemctl start dgiot_lite
```

4 数据库部署

4.1 PostgreSQL

```
# Ubuntu
sudo apt install postgresql postgresql-client -y

# 创建用户和库
sudo -u postgres psql << SQL
CREATE USER iotuser WITH PASSWORD 'iotpass';
CREATE DATABASE iot_platform OWNER iotuser;
GRANT ALL PRIVILEGES ON DATABASE iot_platform TO iotuser;
\q
SQL

# 测试连接
psql -h 127.0.0.1 -U iotuser -d iot_platform -c "SELECT 1"
```

4.2 TDengine

```
# Ubuntu (使用 TDengine 官方仓库)
wget https://docs.tdengine.com/downloads/TDengine-server-3.3.4.0-
Linux-x64.deb
sudo dpkg -i TDengine-server-3.3.4.0-Linux-x64.deb

# 启动
sudo systemctl start taosd taosadapter
sudo systemctl enable taosd taosadapter

# 测试
taos -s "SHOW DATABASES;"
```

4.3 不装数据库？

可以。平台检测不到 PostgreSQL 和 TDengine 时，自动使用 SQLite 存储全部数据，完全零外部依赖。

SQLite 模式适用于：

- 开发测试

- ≤ 50 台设备的小规模场景
- 单机部署

5 协议模拟器

如果暂无真实设备，可使用内置模拟器进行功能验证。

5.1 启动模拟器

```
# 一键启动全部模拟器
python simulators/run_all.py

# 或单独启动
python simulators/modbus_tcp_server.py
python simulators/iec104_server.py
python simulators/opcua_server.py
```

Windows 用户：双击 start_simulators.bat。

5.2 模拟设备清单

协议	端口	模拟设备	关键数据点
Modbus TCP	502	光伏逆变器	电压/电流/功率/发电量
Modbus TCP	1502	储能 PCS	SOC/SOH/充放电功率
Modbus TCP	2502	充电桩	充电功率/电压/电流
IEC 104	2404	储能 PCS	遥测: SOC/功率/温度, 遥信: 运行状态
OPC UA	4840	充电桩 + 环境	充电功率/温度/辐照度

6 DG-IoT 联动配置（可选）

dgiot_lite 可推送数据至 DG-IoT 主平台。配置方式：

6.1 方法一：管理后台配置

1. 登录管理后台（<http://localhost:8000>）
2. 添加推送目标，类型选择 dgiot
3. 填写 DG-IoT MQTT Broker 地址和认证信息

6.2 方法二：API 配置

```
curl -X POST http://localhost:8000/api/push-targets \
  -H "Content-Type: application/json" \
  -d '{
    "target_id": "dgiot_bridge",
    "target_name": "DG-IoT主平台",
    "target_type": "dgiot",
    "endpoint": "192.168.1.100:1883",
    "config": {
      "host": "192.168.1.100",
      "port": 1883,
      "username": "dgiot_admin",
      "password": "dgiot_admin",
      "topic": "dgiot/device/telemetry",
      "product_id": "pcs_monitor"
    }
  }'
```

6.3 验证联动

1. 确认 dgiot_lite 有设备在线采集
2. 登录 DG-IoT 主平台，查看对应产品下的设备数据
3. dgiot_lite 采集数据将自动按 DG-IoT 物模型格式推送

7 部署验证清单

序号	检查项	预期结果	通过
1	curl http://localhost:8000/api/health	{"status":"ok"}	
2	浏览器打开管理后台 http://localhost:8000	正常显示	
3	浏览器打开 http://localhost:8000/docs	API 文档正常显示	
4	添加一台测试设备并配置点位	设备列表中可见	
5	启动模拟器，等待 10 秒	设备状态变为 online	
6	查看仪表盘	采集次数递增	
7	添加 DG-IoT 推送目标（可选）	数据推送至 DG-IoT	

8 常见问题

8.1 端口被占用

```
# 查看端口占用
netstat -tlnp | grep 8000

# 修改 config.yaml 中的 port 字段
```

8.2 数据库连接失败

平台会自动降级 SQLite，不影响使用。如需使用 PostgreSQL:

1. 确认 PostgreSQL 已启动: pg_isready
2. 确认 config.yaml 中连接参数正确
3. 确认数据库已创建: psql -U iotuser -d iot_platform

8.3 模拟器启动报错

```
# Modbus: 确认 pymodbus 已安装
pip install pymodbus

# OPC UA: 确认 asyncua 已安装
pip install asyncua

# 端口被占用: 修改对应脚本中的 port 参数
```

8.4 如何更新平台

```
cd dgiot_lite
git pull origin master
pip install -r requirements.txt
# 重启服务
```